

# 1 / 3 · Attention과 GPT 부품 연결

Q/K/V에서 문맥 벡터를 만들고, cfg로 GPT의 입출력 차원을 연결한다.

실행:  $x \rightarrow Q/K/V \rightarrow \text{score} \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow \text{context}$  | 역추적:  $\text{context} \leftarrow \text{weights} + V$

## [기출 복기] Multi-head Attention의 핵심

```
b, num_tokens, _ = x.shape
keys = self.W_key(x)
queries = self.W_query(x)
values = self.W_value(x)
# Q, K, V 각각에 같은 head 분할을 적용
keys = keys.view(b, num_tokens, self.num_heads,
                  self.head_dim).transpose(1, 2)
queries = queries.view(b, num_tokens, self.num_heads,
                       self.head_dim).transpose(1, 2)
values = values.view(b, num_tokens, self.num_heads,
                     self.head_dim).transpose(1, 2)
attn_scores = queries @ keys.transpose(2, 3)
mask_bool = self.mask.bool()[:num_tokens, :num_tokens]
attn_scores.masked_fill_(mask_bool, -torch.inf)
attn_weights = torch.softmax(
    attn_scores / keys.shape[-1]**0.5, dim=-1)
attn_weights = self.dropout(attn_weights)
context_vec = (attn_weights @ values).transpose(1, 2)
context_vec = context_vec.contiguous().view(
    b, num_tokens, self.d_out)
context_vec = self.out_proj(context_vec)
```

**인자 출처** x: forward 입력 [B,T,Din]. W\_key/query/value-mask-dropout-out\_proj: \_\_init\_\_에서 생성. num\_heads-d\_out: 생성자 입력, head\_dim=d\_out//num\_heads. b-num\_tokens: x.shape.

**실수 방지** head 분할 후 [B,H,T,Dh] → score [B,H,T,T]. softmax는 참고할 토큰 축 -1. 단일 head의 K transpose는 (1,2), multi-head는 (2,3).

근거: Chapter\_3\_Excercise\_Attention.ipynb · CausalAttention / MultiHeadAttention.forward

## [기출 복기] cfg에서 만드는 GPT 입출력

```
self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
self.drop_emb = nn.Dropout(cfg["drop_rate"])
self.out_head = nn.Linear(
    cfg["emb_dim"], cfg["vocab_size"], bias=False)
```

**인자 출처** cfg: GPTModel(cfg)에 전달된 설정 딕셔너리. vocab\_size=ID 종류 수, context\_length=최대 위치 수, emb\_dim=벡터 길이. Attention 생성 시 d\_in/d\_out=emb\_dim, num\_heads=n\_heads, dropout=drop\_rate, qkv\_bias=qkv\_bias.

**실수 방지** 입력 ID [B,T]는 long, 임베딩 [B,T,D]는 float. out\_head의 출력은 [B,T,V] 점수(logits)이며 정답 ID가 아닙니다.

근거: Chapter\_4\_Excercise\_GPT.ipynb · TransformerBlock.\_\_init\_\_ / GPTModel.\_\_init\_\_

가리고 쓰기: QK score의 transpose 축 / softmax 축 / Embedding의 두 인자 / 출력 head의 두 차원.

## 2 / 3 · 정규화, forward, 생성·학습

모델 부품을 실행 순서로 잇고 마지막 토큰 생성과 전체 위치 학습을 구분한다.

실행: ID → embedding → blocks → norm → logits | 역추적: 다음 ID ← 마지막 logits ← GPT

### [범위 핵심] LayerNorm과 GPT forward

```
mean = x.mean(dim=-1, keepdim=True)
var = x.var(dim=-1, keepdim=True, unbiased=False)
norm_x = (x - mean) / torch.sqrt(var + self.eps)
return self.scale * norm_x + self.shift

# GPTModel.forward(self, in_idx)의 핵심
batch_size, seq_len = in_idx.shape
tok_embeddings = self.tok_emb(in_idx)
pos_embeddings = self.pos_emb(
    torch.arange(seq_len, device=in_idx.device))
x = self.drop_emb(tok_embeddings + pos_embeddings)
x = self.trf_blocks(x)
x = self.final_norm(x)
logits = self.out_head(x)
```

**인자 출처** mean-var: 현재 x에서 계산. eps-scale-shift: LayerNorm.\_\_init\_\_. seq\_len: 현재 입력 길이, 최대 길이가 아님. trf\_blocks: \_\_init\_\_의 nn.Sequential 안에 쌓인 TransformerBlock들.

**실수 방지** LayerNorm은 마지막 D축에서 통계 계산, shape 유지. FFN은 Linear(D,4D) → GELU → Linear(4D,D). 블록은 norm→attention→dropout→잔차 더하기, 이어 norm→FFN→dropout→잔차 더하기.

근거: Chapter\_4\_Excercise\_GPT.ipynb · LayerNorm / FeedForward / TransformerBlock / GPTModel

### [기출 복기 + 범위 핵심] 생성과 손실

```
# generate_text_simple(model, idx, max_new_tokens, context_size)
idx_cond = idx[:, -context_size:]
with torch.no_grad():
    logits = model(idx_cond)
logits = logits[:, -1, :]
idx_next = torch.argmax(logits, dim=-1, keepdim=True)
idx = torch.cat((idx, idx_next), dim=1)

# 학습에서는 모든 위치의 정답과 비교
logits = model(input_batch)
loss = torch.nn.functional.cross_entropy(
    logits.flatten(0, 1), target_batch.flatten())
```

**인자 출처** idx-context\_size: 생성 함수 입력. logits: 모델 반환 [B,T,V]. input\_batch-target\_batch: loader 배치 [B,T]. V는 logits의 마지막 축에서 유지.

**실수 방지** 생성: [B,T,V]→[B,V]→[B,1]. 학습 CE: [B\*T,V]와 [B\*T] long. CE 앞에 softmax를 별도로 넣지 않습니다. keepdim=True는 cat에 필요한 배치·토큰 축 유지.

근거: Chapter\_4\_Excercise\_GPT.ipynb · generate\_text\_simple / Chapter\_5\_Excercise\_Pretraining.ipynb · calc\_loss\_batch

가리고 쓰기: (x-평균)/sqrt(분산+eps), 마지막 위치 선택, [B,1] argmax, CE의 두 flatten.

## 3 / 3 · Dataset, 분류, LoRA·DPO

학습 목적을 보고 입력·정답·학습 파라미터를 구성한다.

실행: 문장 → ID → window/padding → batch → loss | 역추적: loss ← logits + 목적에 맞춘 target

### [범위 핵심] 다음 토큰과 분류의 정답 구성

```
# GPTDatasetV1: 함수/생성자 입력 token_ids, max_length, stride
input_chunk = token_ids[i:i + max_length]
target_chunk = token_ids[i + 1:i + max_length + 1]
# Instruction collate: 배치의 최대 길이로 padding한 padded
inputs = torch.tensor(padded[:-1])
targets = torch.tensor(padded[1:])
# 분류 데이터: 먼저 최대 길이로 자르고 남은 길이만 padding
encoded_text = encoded_text[:self.max_length]
encoded_text = encoded_text + [pad_token_id] * (
    self.max_length - len(encoded_text))
```

**인자 출처** i: window 반복문. max\_length·stride: Dataset 생성자. pad\_token\_id: collate/데이터셋 함수 입력. self.max\_length: 설정값 또는 학습 데이터 최대 길이. labels: 데이터의 Label 열.

**실수 방지** next-token target도 [B,T]입니다. 분류 label은 [B]. 분류 head는 Linear(emb\_dim,num\_classes), loss에는 logits[:, -1, :] 사용. 반복 padding target은 ignore\_index로 제외하되 EOS 학습 여부를 원본대로 유지.

근거: Chapter\_2\_Exercise\_Dataset / Chapter\_6\_Exercise\_Finetuning\_Classification / Chapter\_7\_Exercise\_Follow\_Instructions

### [기출 복기] LoRA 행렬과 DPO 선호 방향

```
# LoRALayer.__init__(in_dim, out_dim, rank, alpha)
self.A = nn.Parameter(torch.empty(in_dim, rank))
self.B = nn.Parameter(torch.zeros(rank, out_dim))
# LoRALayer.forward: 기존 Linear에 더할 보정값
delta = (self.alpha / self.rank) * (x @ self.A @ self.B)
# 결합 출력: self.linear(x) + self.lora(x)

# DPO: policy와 reference에서 얻은 응답 log-probability
model_logratios = model_chosen_logprobs - model_rejected_logprobs
reference_logratios = (reference_chosen_logprobs
    - reference_rejected_logprobs)
logits = model_logratios - reference_logratios
losses = -F.logsigmoid(beta * logits)
```

**인자 출처** in\_dim·out\_dim: 원본 Linear의 in\_features/out\_features. rank·alpha: LoRA 설정. DPO의 네 logprob와 beta: compute\_dpo\_loss 함수 입력. prompt/chosen/rejected는 같은 데이터 항목에서 가져옵니다.

**실수 방지** A는 무작위 초기화, B는 0으로 시작해 초기 보정을 0으로 만듭니다. 기본 weight 고정, LoRA 학습. DPO는 chosen-rejected 방향이며 reference는 고정합니다. LoraConfig가 주어진다면 r·lora\_alpha·target\_modules를 설정값에서 대응시킵니다.

근거: Chapter\_6\_Finetuning\_for\_Text\_Classification\_LoRA\_KR.ipynb · LoRALayer / Chapter\_7\_Exercise\_Follow\_Instructions\_dpo.ipynb · compute\_dpo\_loss

가리고 쓰기: window 끝과 target 끝의 차이 / 패딩 개수 / LoRA A·B shape / chosen-rejected 방향.